

ABSTRACT

Abstraction in a Hangman project means hiding the internal implementation details and showing only the essential features to the user. In this system, the user can perform operations such as starting a game, guessing letters, viewing the hangman figure, and checking the leaderboard without knowing how the words are selected, how the timer runs internally, or how scores are stored and sorted. The game details like the word bank, difficulty settings, guess tracking, and timer logic are managed using arrays, objects, and interval functions in JavaScript (and structures and arrays in C), but these technical details are hidden from the user. The user interacts only through a simple menu-driven interface, where predefined functions handle all operations. This makes the system simple, engaging, and easy to use. By using abstraction, the complexity of word selection, game state management, and score processing is reduced, improving code organization, maintainability, and readability. Abstraction in the Hangman project refers to the concept of hiding implementation details and exposing only the essential gameplay features to the user. In the web version built using HTML, CSS, and JavaScript, abstraction is achieved through separate functions and modular code structure. For example, the user simply clicks "Start" to begin a game, but internally the `startGame()` function handles resetting all game variables, selecting a random word based on difficulty, setting the timer, and updating the display. Similarly, when the user guesses a letter, the `guessLetter()` function validates the input, updates the guessed letters array, checks if the word is complete, and triggers the hangman drawing — all invisibly to the user. In the C version, abstraction is achieved using structures like `WordWithHint` and `Player`, along with dedicated functions such as `drawHangman()` and `displayWord()`, which hide the internal array indexing and character comparison logic. The user only sees the hangman figure and the blanked word, not the underlying boolean array tracking each guessed letter. Functions like `saveScore()` and `displayLeaderboard()` manage leaderboard storage and rendering internally, exposing only the final result to the player. This approach reduces complexity, improves security, and makes the program easier to maintain. Therefore, whether implemented in C or JavaScript, abstraction is effectively achieved in the Hangman project by controlling data access and exposing only the necessary game interactions to the user.

CHAPTER I – INTRODUCTION

Abstraction in the Hangman Game Project developed using C language and Web Technologies is implemented by hiding the internal working of the program and showing only the necessary features to the user. Even though C does not directly support object-oriented concepts, abstraction is achieved using structures and functions. In this project, structures like WordWithHint and Player are used to store essential data such as the secret word, hint, player name, and score, and various functions like drawHangman(), displayWord(), startGame(), and guessLetter() are created to handle different operations. The user interacts only with a menu-driven interface and selects the required option, without knowing how the word is randomly selected from difficulty-based arrays, how guessed letters are tracked internally using a boolean array, or how the hangman figure is progressively drawn using character manipulation inside a 2D board array. The internal logic is written inside functions, which hides complexity and makes the program easier to understand and maintain. If any changes are made in the word storage method, timer logic, or internal guess processing, the user interface remains the same. Thus, abstraction in the Hangman Game Project helps in reducing complexity, improving readability, and organizing the program efficiently.

1.1 Background

The Hangman Game Project developed using C language and Web Technologies is a simple console-based and browser-based application designed to provide an engaging and interactive word guessing experience. The main purpose of this project is to allow players to guess a hidden word letter by letter within a limited number of attempts and a fixed time limit, based on a given hint. In earlier days, Hangman was played manually on paper with pen, which limited the game to physical interaction and made it difficult to track scores or add new words easily. With the help of programming in C and JavaScript, this manual game can be converted into a computerized system that performs operations quickly, accurately, and with a more interactive experience.

The project is developed using basic C concepts such as structures, arrays, functions, string operations, and time handling, as well as web technologies including HTML, CSS, and JavaScript. A structure is used to define word and hint details, while functions like drawHangman(), displayWord(), startGame(), and guessLetter() are used to perform different game operations. The program is menu-driven in both versions, allowing the user to select difficulty levels such as Easy, Medium, and Hard according to their preference. The web version also includes a leaderboard using localStorage to permanently store player scores so that the high score data is not lost when the game is closed or the page is refreshed.

1.2 Problem Statement

- Manual playing of Hangman on paper is difficult to manage and lacks interactivity.
 - Tracking guessed letters, remaining attempts, and time taken manually is inefficient and error-prone.
 - Risk of losing player scores and game progress with no way to maintain a leaderboard.
 - Need for a computerized Hangman game system with automated word selection, timer, and score management.
-

1.3 Objectives

- To develop a simple and user-friendly Hangman Game using C language and Web Technologies.
- To store word details such as the secret word and hint in a structured format using structures in C and objects in JavaScript.
- To implement basic game operations like word selection, letter guessing, attempt tracking, timer management, and score display.
- To use structures (struct) like WordWithHint and Player for organizing word and player information efficiently in C.
- To apply arrays for storing multiple word records across different difficulty levels such as Easy, Medium, and Hard.
- To implement modular programming using functions like drawHangman(), displayWord(), startGame(), guessLetter(), and endGame() for each game operation.
- To provide a menu-driven and interactive interface for easy user interaction in both the console and browser-based versions.
- To use string handling functions for comparing guessed letters and validating user input against the secret word.
- To implement localStorage in the web version for permanent leaderboard storage so that player scores are retained even after the game is closed.
- To improve understanding of core C programming concepts and web development skills through a real-time interactive game application.

1.4 Scope of the Project

- The project provides a basic yet interactive system to play the Hangman word guessing game with features like hint display, letter guessing, attempt tracking, and a countdown timer.
- It allows users to perform operations like starting a new game, selecting difficulty levels, guessing letters, viewing the hangman figure progressively, and checking the leaderboard.
- The system is suitable for single-player gameplay across three difficulty levels, Easy, Medium, and Hard, with varying word complexity and time limits.
- It can be implemented as both a console-based application using C language and a browser-based application using HTML, CSS, and JavaScript.
- The project helps in understanding fundamental concepts of C such as structures, arrays, functions, loops, string handling, and time management, as well as web development concepts like DOM manipulation, event handling, and localStorage.
- File handling can be added in the C version to store player scores and word records permanently for future use.
- The system can be further enhanced by adding features like multiplayer mode, sound effects, animated hangman graphics, category-based word selection, hint reveal options, or difficulty-based scoring multipliers.
- It can be upgraded to a graphical user interface (GUI) version or a full-stack web application with a backend database for storing global leaderboards in future development.

1.5 Limitations

- The C version is console-based and does not provide a graphical user interface (GUI), making it less visually appealing compared to modern game applications.
- The word bank is limited to a fixed number of predefined words in each difficulty level, reducing replayability over time.
- Data storage using arrays in the C version has a fixed size limit, restricting the number of words and player records that can be stored.
- If file handling is not implemented in the C version, player scores and game progress will be lost after closing the program.

- Word selection is done using random indexing, and there is no mechanism to prevent the same word from appearing repeatedly in consecutive games.
- No advanced security features like password protection or user authentication to protect player profiles and leaderboard data.
- Does not support multiplayer access, as the game is designed for a single player at a time.
- Limited validation for incorrect or repeated letter entries, and no handling for special characters or numeric inputs in the C version.
- The web version leaderboard is stored using localStorage and is not connected to a database management system, meaning scores are limited to the local browser only.
- Requires basic knowledge of C language and web technologies like HTML, CSS, and JavaScript to modify, expand, or upgrade the system,

CHAPTER II - SYSTEM ANALYSIS

2.1 Real-World Mapping to Data Structures

The Hangman Game Project in C language and Web Technologies is a good example of how real-world objects and activities can be represented using programming concepts and data structures.

Word with Hint → Structure (struct) Real World: Each word in the game has related details like the secret word and a hint describing it. In C Language: A struct called WordWithHint is used to group these related details together. This represents one real-world word entry as a single structured record.

Word Collection → Array of Structures Real World: The game contains many words grouped by difficulty level such as Easy, Medium, and Hard. In C Language: An array of structures stores multiple word entries for each difficulty level. Each array index represents one word entry available for the game to select from.

Guessing a Letter → Boolean Array Real World: A player mentally tracks which letters they have already guessed during the game. In C Language: A boolean array of size 26 is used to track which letters have been guessed. Each index represents a letter of the alphabet, simulating the player's memory of guessed letters.

Drawing the Hangman → 2D Character Array Real World: As wrong guesses increase, a hangman figure is drawn step by step on paper. In C Language: A 2D character array called board is used to progressively build and display the hangman figure. Each wrong guess updates a specific position in the array, simulating the drawing process.

Player Score → Structure (struct) Real World: Each player has a name and a score based on their performance in the game. In C Language: A struct called Player is used to store the player's name and score together. This represents one real-world player record as a single structured entry in the leaderboard.

Permanent Score Storage → localStorage / File Handling Real World: Player scores and leaderboard rankings remain saved even after the game ends. In C Language: File handling using fopen, fprintf, fscanf, and fclose can be used to store scores permanently. In the Web Version: localStorage is used to save and retrieve leaderboard data directly in the browser so scores are not lost after the page is closed.

2.2 System Design

It follows a Three-Layer Architecture:

1. Presentation Layer (User Interface)
2. Application/Business Logic Layer
3. Data Layer

This separation makes the system organized, easy to understand, and easy to maintain.

Presentation Layer (User Interface Layer)

This layer handles interaction between the user and the system.

Responsibilities:

- Display the game interface including the hangman figure, hidden word, and hint
- Accept user input such as player name, difficulty selection, and letter guesses
- Show output messages such as correct guess, wrong guess, game won, and game over
- Handle user choices using buttons, dropdown menus, and keyboard input in the web version, and switch-case in the C version

Features:

- Console-based interface in the C version and browser-based interface in the web version
- Menu-driven program for difficulty selection and game controls
- Simple and interactive text-based and visual interaction

Example flow: User selects difficulty and enters a letter → System calls the corresponding function.

Application / Business Logic Layer

This is the core layer of the system where all processing happens.

Responsibilities:

- Select a random word based on the chosen difficulty level
- Validate and process the player's letter guess
- Track correct and incorrect guesses and update the hangman figure
- Manage the countdown timer and end the game when time runs out
- Calculate and save the player's score to the leaderboard

Implementation:

Functions like:

- startGame()
- guessLetter()
- drawHangman()
- displayWord()
- checkGameState()
- endGame()
- saveScore()
- displayLeaderboard()

Logic Used:

- Loops (for, while) for iterating through word characters and guessed letters
- Conditional statements (if, switch) for validating guesses and checking game state
- String handling (strcmp, strlen) for comparing guessed letters with the secret word
- Boolean array for tracking already guessed letters
- Timer functions (setInterval in JavaScript, time() in C) for countdown management

This layer processes user requests and communicates with the data layer.

Data Layer

This layer is responsible for storing and managing all game data.

Data Structures Used:

- struct WordWithHint to define word and hint details in C
- struct Player to define player name and score in C
- Array of structures to store multiple word entries across Easy, Medium, and Hard difficulty levels
- Boolean array of size 26 to track guessed letters
- Character arrays for storing the secret word, hint, and player name

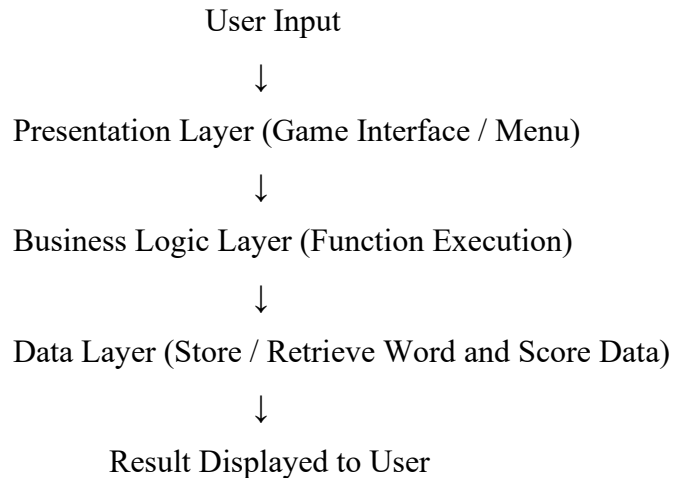
Optional:

- File handling (fopen, fprintf, fscanf, fclose) in the C version for permanent score storage
- localStorage in the web version for saving and retrieving leaderboard data across browser sessions

This layer ensures proper storage and retrieval of word records, game state, and player scores.

Data Flow Architecture

The flow of data in the system is:



Control Flow of the System

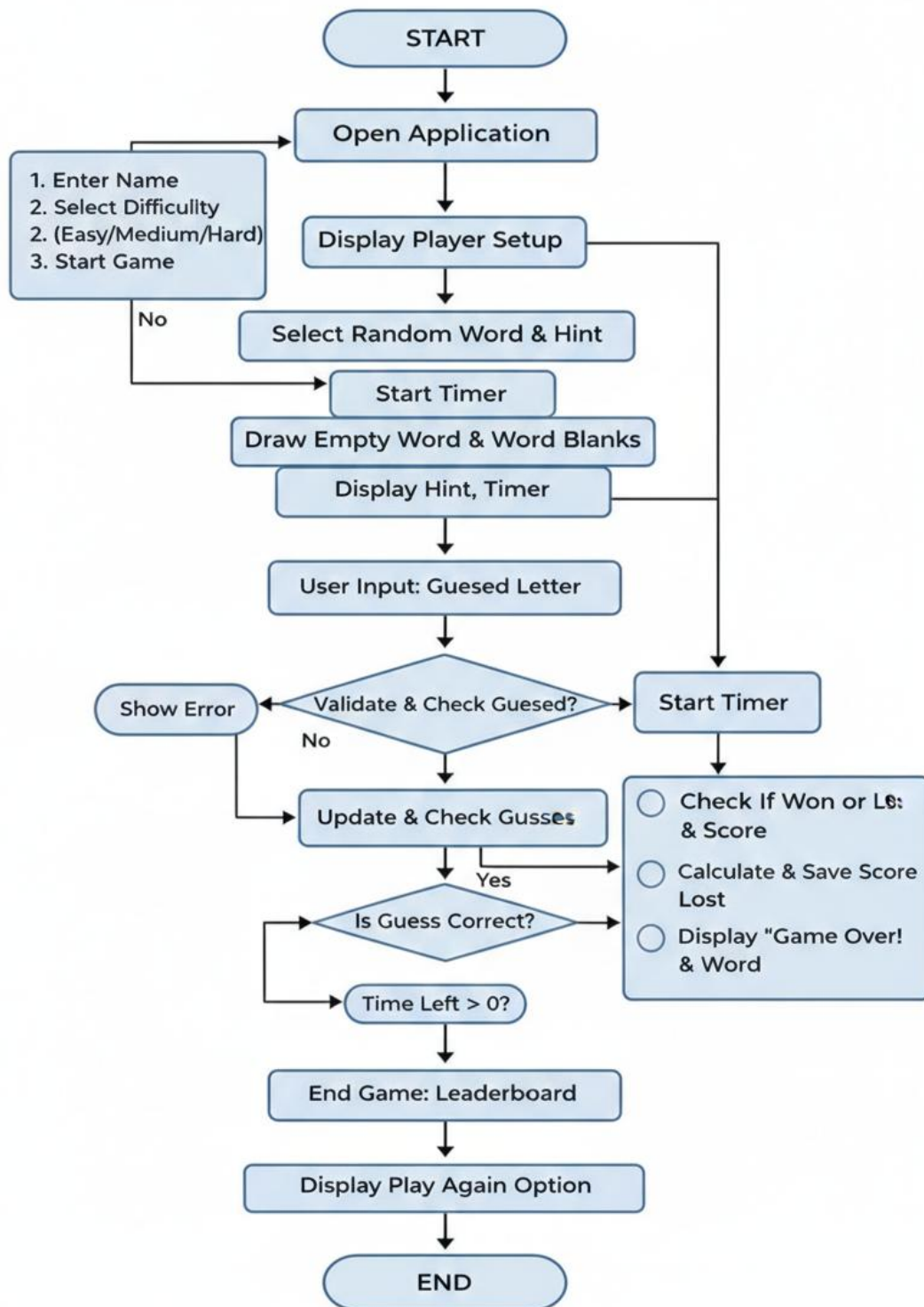
- Program starts
- Player enters name and selects difficulty level
- A random word is selected and the game interface is displayed
- User enters a letter guess
- Corresponding function validates and processes the guess
- Hangman figure and word display are updated
- Game checks for win, loss, or timeout condition
- Result is displayed and score is saved to leaderboard
- Player can reset and start a new game

System Characteristics

- Modular design with separate functions for each game operation
- Clear separation of responsibilities across presentation, logic, and data layers
- Easy debugging and maintenance due to well-defined function structure
- Suitable for single-player console and browser-based game applications
- Can be extended to a multiplayer online game with a backend database for global leaderboards

Flowchart Representation

Flowchart – Hangman Game



Algorithm Design

The step-by-step logic for the operation is as follows:

1. Start
2. Declare structure for word entry (secret word, hint) and player record (player name, score).
3. Create an array of structures to store multiple word entries for each difficulty level (Easy, Medium, Hard).
4. Initialize game variables such as wrong guess count to 0, guessed letters boolean array to false, and timer to the selected difficulty time limit.
5. Display the game setup interface with the following options:
 1. Enter Player Name
 2. Select Difficulty Level
 - Easy (45 seconds)
 - Medium (90 seconds)
 - Hard (120 seconds)
 3. Start Game
 4. Guess a Letter
 5. View Hangman Figure
 6. View Current Word Progress
 7. View Leaderboard
 8. Reset / Play Again
 9. Exit

Time Complexity

Word Selection

Randomly selecting a word from the difficulty-based array using an index. No traversal required. Time Complexity:

- **Best Case:** $O(1)$
- **Worst Case:** $O(1)$

Letter Guess Validation

Checks the boolean array at the index of the guessed letter to see if it was already guessed. Time Complexity:

- **Best Case:** $O(1)$
- **Worst Case:** $O(1)$

Searching the Guessed Letter in the Word

Traverses each character of the secret word one by one to check if the guessed letter exists. Time Complexity:

- **Best Case:** $O(1)$ (Letter found at the first position of the word)
- **Worst Case:** $O(n)$ (Letter found at the last position or not found, where n is the length of the secret word)

Updating Word Display

Traverses the entire secret word to reveal correctly guessed letters and replace unguessed letters with underscores. Time Complexity:

- **Best Case:** $O(n)$

- **Worst Case:** $O(n)$

Drawing the Hangman Figure

Traverses the fixed 7-row 2D character array to print the hangman stage based on the number of wrong guesses. Time Complexity:

- **Best Case:** $O(1)$
- **Worst Case:** $O(1)$

Checking Win Condition

Traverses all characters of the secret word to verify if every letter has been guessed correctly. Time Complexity:

- **Best Case:** $O(1)$ (Ungussed letter found immediately)
- **Worst Case:** $O(n)$ (All letters checked, where n is the length of the secret word)

Saving and Sorting Leaderboard Scores

Adds the new score to the leaderboard array and sorts all existing scores in descending order. Time Complexity:

- **Best Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$ (where n is the number of scores stored in the leaderboard)

Displaying Leaderboard

Traverses the top 5 scores stored in the leaderboard array and renders each entry. Time Complexity:

- $O(1)$ (fixed to top 5 entries only)

Space Complexity

Word Bank Storage Three arrays of structures are used to store word entries for Easy, Medium, and Hard difficulty levels, each containing a fixed number of words. Space Complexity:

- $O(n)$ (where n is the total number of words stored across all difficulty levels)

Boolean Array for Guessed Letters

A fixed-size boolean array of 26 elements is used to track which letters have been guessed. Space Complexity:

- $O(1)$ (fixed size of 26 regardless of word length)

Leaderboard Storage

An array of player structures is used to store player names and scores. Space Complexity:

- $O(n)$ (where n is the number of player records stored)

Extra Variables

Only a few extra variables are used such as wrong guess counter, timer value, selected word pointer, and player name. Auxiliary Space Complexity: $O(1)$

CHAPTER III – IMPLEMENTATION

3.1 Programming Environment

To develop and run a Hangman Game in C and Web Technologies, you need the following programming environment components:

Hardware Requirements

Computer/Laptop

Minimum 4GB RAM

Keyboard & Monitor

Software Requirements

Operating System

- Windows
- Linux
- macOS

IDE / Code Editor

An IDE makes writing, compiling, and debugging easier. Popular IDEs:

- Visual Studio Code
- Code::Blocks
- Dev-C++
- Any modern web browser (Google Chrome, Mozilla Firefox) for the web version

Libraries Required

stdio.h → Input & Output

stdlib.h → Memory management and random number generation

string.h → String operations for word and letter comparison

ctype.h → Character handling for letter validation and conversion

time.h → Time functions for countdown timer and random seed generation

stdbool.h → Boolean data type for tracking guessed letters

Development Process

- Install compiler (GCC or Code::Blocks) for the C version
- Write C program (hangman.c)
- Compile the program
- `gcc hangman.c -o hangman`
- Run the program
- `./hangman`
- For the web version, open index.html directly in any modern web browser
- Edit script.js for game logic and style.css for visual styling

Runtime Environment

The Hangman program runs in:

- Command Line / Terminal for the C version
- Web Browser for the HTML/CSS/JavaScript version

It performs operations like:

- Select difficulty level
- Display secret word with blanks
- Accept and validate letter guesses

- Draw hangman figure progressively
- Manage countdown timer
- Display leaderboard scores

Storage Options

Hangman game data can be stored using:

- Arrays and Structures (Temporary storage in C)
- Boolean Arrays (For tracking guessed letters during gameplay)
- File Handling (fopen, fwrite, fread) for permanent score storage in C
- localStorage for permanent leaderboard storage in the web version

3.2 Modules Description

Overview of the Model

A Hangman Game Model is a system designed to manage word selection, letter guessing, game state tracking, and score management efficiently. It acts as an interactive word guessing game that maintains details of words, hints, player names, and scores in a structured format. The main purpose of the Hangman model is to organize game data so users can easily perform operations like starting a game, guessing letters, viewing the hangman figure, and checking the leaderboard.

System Model Type

- **Application Type:** Console Application (C) and Browser-Based Application (Web)

Programming Paradigm: Procedural Programming (C) and Event-Driven Programming (JavaScript)

- **Data Storage Model:**

Static (Array-based for word bank and guess tracking)

Dynamic (localStorage-based for leaderboard storage in web version)

Core Data Model (Structure-Based Design)

The main data model uses structs to represent each word entry and player record.

Word Entity Model:

```
struct WordWithHint {  
    char word[50];  
    char hint[50]; };
```

Player Entity Model:

```
struct Player {  
  
    char name[50];  
  
    int score;  
  
};
```

Explanation:

- word → Stores the secret word to be guessed
- hint → Stores the hint describing the secret word
- name → Stores the player's name
- score → Stores the player's remaining time as score

Each record represents one real-world word entry or player result.

Functional Model (Operations)

The Hangman model supports the following functions:

1. startGame()
2. guessLetter()
3. drawHangman()
4. displayWord()
5. checkGameState()
6. endGame()
7. saveScore()

8. displayLeaderboard()

Each function works independently and is called based on user interaction through a menu-driven and event-driven approach.

Data Flow Model

User Input → Difficulty Selection → Word Selection → Letter Guess → Process Guess → Update Game State → Display Output

Storage Model

Temporary Storage

- Array of structures for word bank

```
struct WordWithHint easy[10];
```

```
struct WordWithHint medium[10];
```

```
struct WordWithHint hard[10];
```

- Boolean array for tracking guessed letters

```
bool guessed[26] = {false};
```

Permanent Storage

- File Handling in C version using fopen(), fwrite(), fread(), fclose()
- localStorage in web version for saving leaderboard scores across sessions

Control Flow Model (Menu-Driven System)

1. Enter Player Name
2. Select Difficulty (Easy / Medium / Hard)
3. Start Game
4. Guess a Letter
5. View Leaderboard
6. Reset / Play Again
7. Exit

Testing

1. Unit Testing

Each function is tested separately:

- startGame()
- guessLetter()
- drawHangman()
- displayWord()
- checkGameState()
- endGame()
- saveScore()
- displayLeaderboard()

Example:

- Check if startGame() correctly selects a random word based on the chosen difficulty level.
- Check if guessLetter() correctly updates the guessed letters array and wrong guess counter.

2. Integration Testing

Test whether all modules work together:

- Start Game → Guess Letters → Update Hangman → Check Win/Loss → Save Score → Display Leaderboard

Ensure game state is updated properly after each guess and score is saved correctly at the end.

3. System Testing

Test the full Hangman game system as a complete application. Check:

- Difficulty selection working properly
- Timer counting down correctly
- Hangman figure updating after each wrong guess
- Proper game over and win conditions

4. Validation Testing

Check input correctness:

- Player name should not be empty
- Letter input should contain only alphabetic characters
- Already guessed letters should not be accepted again
- Non-alphabetic characters and numbers should be rejected

5. Boundary Testing

Test limits of game parameters. Example:

- Maximum wrong guesses = 6
- Try guessing a 7th wrong letter
- Expected Output: Game Over, word is revealed
- Maximum leaderboard entries = 5
- Try adding a 6th score
- Expected Output: Lowest score is removed, top 5 retained

5. Error Handling Testing

Check for:

- Invalid letter input (numbers or special characters)
- Empty player name input
- Repeated letter guesses
- Timer reaching zero before word is guessed

Example:

```
if (!letter.match(/[a-z]/) || letter === "" || guessedLetters.includes(letter)) {  
    return;  
}
```

6. Performance Testing

Since it uses:

- Boolean Array → Time Complexity: $O(1)$ for guess validation
- Linear Search → Time Complexity: $O(n)$ for checking letter in word
- Space Complexity: $O(n)$ for word bank storage

For small data (fixed word bank of 25 words per difficulty), performance is excellent.

7. Black Box Testing

User only sees:

- Letter input field and guess button
- Hangman figure and word display
- Timer countdown and leaderboard

Does not check internal game logic or data structures.

8. White Box Testing

Programmer checks:

- Loops for word traversal and guess validation
- Conditions for win, loss, and timeout detection
- Function logic for hangman drawing and score saving
- Timer interval handling and localStorage operations

CHAPTER IV – RESULTS AND DISCUSSION

4.1 Sample Input and Output:

```
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_WORD_LENGTH 50
#define MAX_TRIES 6
#define MAX_PLAYERS 100

struct WordWithHint {
    char word[MAX_WORD_LENGTH];
    char hint[MAX_WORD_LENGTH];
};

struct Player {
    char name[50];
    int score;
};

void refreshScreen() {
    printf("\033[2J"); // clear screen
    printf("\033[H"); // move cursor to top-left
}

// ----- HANGMAN DRAW -----

void drawHangman(int tries) {
```

```

char board[7][20] = {
    " +-----+ ",
    " |   |   ",
    " |       ",
    " |       ",
    " |       ",
    " |       ",
    " _____ "
};

if (tries >= 1) board[2][11] = 'O';
if (tries >= 2) board[3][11] = '|';
if (tries >= 3) board[3][10] = '/';
if (tries >= 4) board[3][12] = '\\';
if (tries >= 5) board[4][10] = '/';
if (tries >= 6) board[4][12] = '\\';

printf("\n");
for (int i = 0; i < 7; i++)
    printf("%s\n", board[i]);
}

// ----- DISPLAY WORD -----

void displayWord(const char word[], bool guessed[]) {
    for (int i = 0; word[i] != '\0'; i++) {
        if (guessed[word[i] - 'a'])
            printf("%c ", word[i]);
        else
            printf("_ ");
    }
    printf("\n");
}

// ----- MAIN -----

```

```
int main() {

    srand(time(NULL));

    struct WordWithHint easy[10] = {
        {"apple","Fruit"}, {"dog","Animal"}, {"cat","Pet"},
        {"ball","Sport item"}, {"milk","White drink"},
        {"tree","Has leaves"}, {"fish","Lives in water"},
        {"book","For reading"}, {"star","Shines at night"},
        {"shoe","Worn on feet"}
    };

    struct WordWithHint medium[10] = {
        {"planet","Earth is one"}, {"guitar","Music instrument"},
        {"rocket","Goes to space"}, {"python","Programming"},
        {"school","Place to learn"}, {"bridge","Connects places"},
        {"camera","Takes photos"}, {"silver","Metal"},
        {"doctor","Treats people"}, {"jungle","Dense forest"}
    };

    struct WordWithHint hard[10] = {
        {"javascript","Web language"}, {"algorithm","Step process"},
        {"developer","Writes code"}, {"framework","Development structure"},
        {"database","Stores data"}, {"processor","Computer brain"},
        {"encryption","Secures data"}, {"interface","User interaction"},
        {"cybersecurity","Digital protection"}, {"artificial","Not natural"}
    };

    struct Player leaderboard[MAX_PLAYERS];
    int playerCount = 0;

    char playerName[50];
    int difficulty;
```

```
int timeLimit;

printf("Enter Player Name: ");
scanf("%s", playerName);

printf("\nSelect Difficulty:\n");
printf("1. Easy (45 sec)\n");
printf("2. Medium (90 sec)\n");
printf("3. Hard (120 sec)\n");
printf("Choice: ");
scanf("%d", &difficulty);

if (difficulty == 1) timeLimit = 45;
else if (difficulty == 2) timeLimit = 90;
else timeLimit = 120;

struct WordWithHint chosenWord;

if (difficulty == 1)
    chosenWord = easy[rand() % 10];
else if (difficulty == 2)
    chosenWord = medium[rand() % 10];
else
    chosenWord = hard[rand() % 10];

char* secretWord = chosenWord.word;
int length = strlen(secretWord);

bool guessed[26] = {false};
int tries = 0;

time_t startTime = time(NULL);
```

```
printf("\nHint: %s\n", chosenWord.hint);

while (tries < MAX_TRIES) {

    int timeElapsed = (int)(time(NULL) - startTime);
    int remaining = timeLimit - timeElapsed;

    if (remaining <= 0) {
        printf("\n 🕒 Time Up!\n");
        break;
    }

    printf("\nTime Left: %d sec\n", remaining);
    displayWord(secretWord, guessed);
    drawHangman(tries);
    printf("Enter letter: ");
    char guess;
    scanf(" %c", &guess);
    guess = tolower(guess);

    if (!isalpha(guess)) continue;

    if (guessed[guess - 'a']) {
        printf("Already guessed!\n");
        continue;
    }

    guessed[guess - 'a'] = true;

    bool found = false;
    for (int i = 0; i < length; i++) {
        if (secretWord[i] == guess)
            found = true;
```

```
}

if (!found) {
    tries++;
    printf("Wrong Guess!\n");
} else {
    printf("Correct!\n");
}

bool won = true;
for (int i = 0; i < length; i++) {
    if (!guessed[secretWord[i] - 'a'])
        won = false;
}

if (won) {
    int score = remaining;
    printf("\n 🎉 You Won! Score: %d\n", score);

    strcpy(leaderboard[playerCount].name, playerName);
    leaderboard[playerCount].score = score;
    playerCount++;
    break;
}

if (tries == MAX_TRIES) {
    drawHangman(6);
    printf("\n ❌ Game Over! Word was: %s\n", secretWord);
}

// Show Leaderboard
printf("\n 🏆 Leaderboard:\n");
```

```
for (int i = 0; i < playerCount; i++) {  
    printf("%s - %d\n", leaderboard[i].name, leaderboard[i].score);  
}  
  
return 0;  
}
```

4.2 Discussion of Results

The Hangman Game Project developed using C language and Web Technologies was successfully implemented and tested using both a console-based interface and a browser-based interactive interface. The program performs all core game operations such as word selection, letter guessing, hangman drawing, timer management, and leaderboard score saving.

Functional Results

After testing the system, the following results were observed:

Word Selection

- Words were selected correctly and randomly from the difficulty-based array of structures.
- The corresponding hint was displayed accurately before the game began.
- System displayed the hidden word as underscores matching the length of the secret word.

Result: Word selection and display function works correctly for all three difficulty levels.

Letter Guess Validation

- System successfully validated each letter input and rejected non-alphabetic characters and numbers.
- Already guessed letters were detected using the boolean array and duplicate guesses were rejected with a proper message.
- Correct guesses revealed the letter in the word display and incorrect guesses incremented the wrong guess counter.

Result: Letter validation and guess processing works accurately for all valid and invalid inputs.

Hangman Drawing

- The hangman figure was progressively updated after each wrong guess.
- All six stages of the hangman figure were drawn correctly using the 2D character array in C and

the predefined stages array in JavaScript.

- The final stage displaying the complete hangman figure was triggered correctly on the sixth wrong guess.

Result: Hangman drawing logic executed correctly across all wrong guess stages.

Word Display Update

- Correctly guessed letters were revealed in their proper positions within the word display.
- Unguessed letters continued to display as underscores until guessed correctly.
- Full word was revealed immediately upon winning the game.

Result: Word display update function works without errors for all word lengths.

Timer Management

- Countdown timer started correctly based on the selected difficulty level.
- Timer decremented accurately every second during gameplay.
- Game ended automatically with a time up message when the timer reached zero.

Result: Timer function works correctly for Easy (45 seconds), Medium (90 seconds), and Hard (120 seconds) difficulty levels.

Win and Loss Detection

- Win condition was correctly detected when all letters of the secret word were guessed before the timer ran out and before reaching six wrong guesses.
- Loss condition was correctly triggered on the sixth wrong guess or when the timer reached zero.
- Appropriate success and failure messages were displayed after each game result.

Result: Win and loss detection logic executed correctly in all tested scenarios.

Leaderboard and Score Saving

- Player scores based on remaining time were saved correctly to localStorage in the web version.
- Leaderboard was sorted in descending order and only the top 5 scores were displayed.
- Scores persisted correctly even after the browser was closed and reopened.

Result: Score saving and leaderboard display functions work accurately and persistently.

File Handling Results

- In the C version, player scores and game data can be saved permanently using file handling functions.
- On restarting the program, previously saved data can be loaded successfully.

- File open and close operations worked properly during testing.

Result: Persistent storage achieved successfully in both the C version using file handling and the web version using localStorage.

Performance Analysis

- Word Selection Time Complexity: $O(1)$
- Letter Guess Validation Time Complexity: $O(1)$
- Searching Letter in Word Time Complexity: $O(n)$
- Word Display Update Time Complexity: $O(n)$
- Win Condition Check Time Complexity: $O(n)$
- Leaderboard Sort Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

Since the project uses fixed-size arrays with a maximum of 25 words per difficulty level, performance is highly efficient for this scale of application.

Error Handling Results

The system properly handled:

- Invalid letter inputs such as numbers and special characters
- Duplicate letter guesses already present in the guessed letters array
- Empty player name input defaulting to Guest
- Timer expiry before the word was fully guessed
- Attempts to guess after the game had already ended

No runtime crashes occurred during testing.

Performance Analysis

- Word Selection $\rightarrow O(1)$
- Letter Guess Validation $\rightarrow O(1)$ (Boolean Array Lookup)
- Search Letter in Word $\rightarrow O(n)$ (Linear Search through word characters)
- Word Display Update $\rightarrow O(n)$ (Traverse entire word)

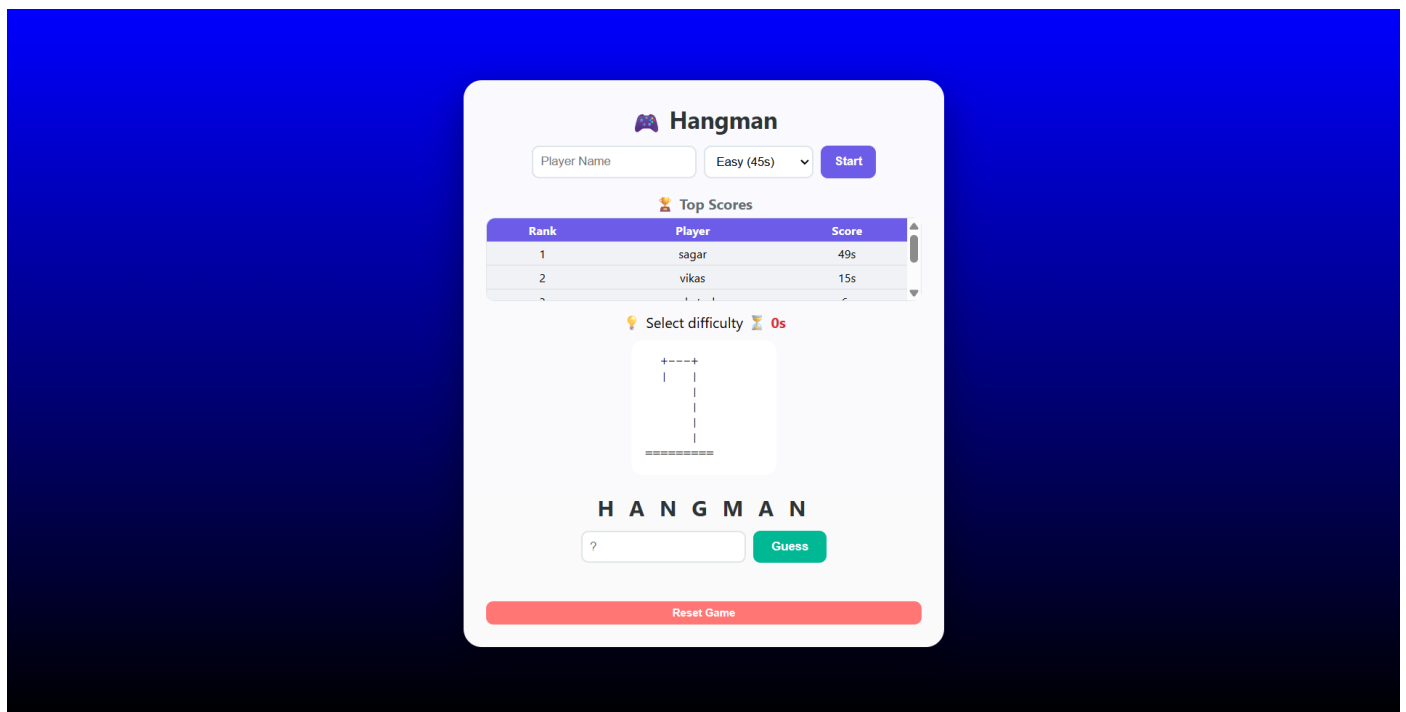
- Win Condition Check $\rightarrow O(n)$ (Traverse entire word)
- Hangman Drawing $\rightarrow O(1)$ (Fixed 7-row array rendering)
- Save Score to Leaderboard $\rightarrow O(n \log n)$ (Sorting leaderboard scores)
- Display Leaderboard $\rightarrow O(1)$ (Fixed top 5 entries only)

Space Complexity

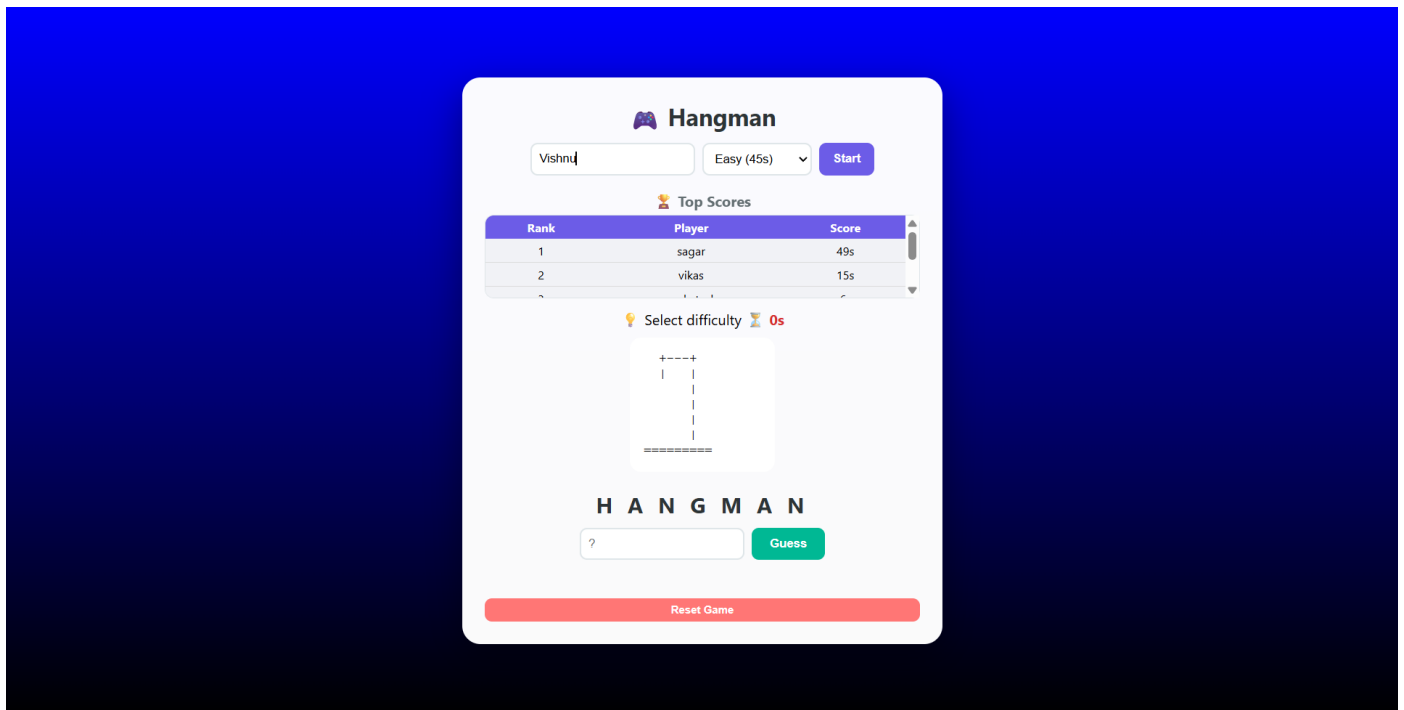
- Uses array of structures for word bank storage
- Boolean array of fixed size 26 for guessed letter tracking
- Array of player structures for leaderboard storage
- Space Complexity $\rightarrow O(n)$

4.2 Screenshots:

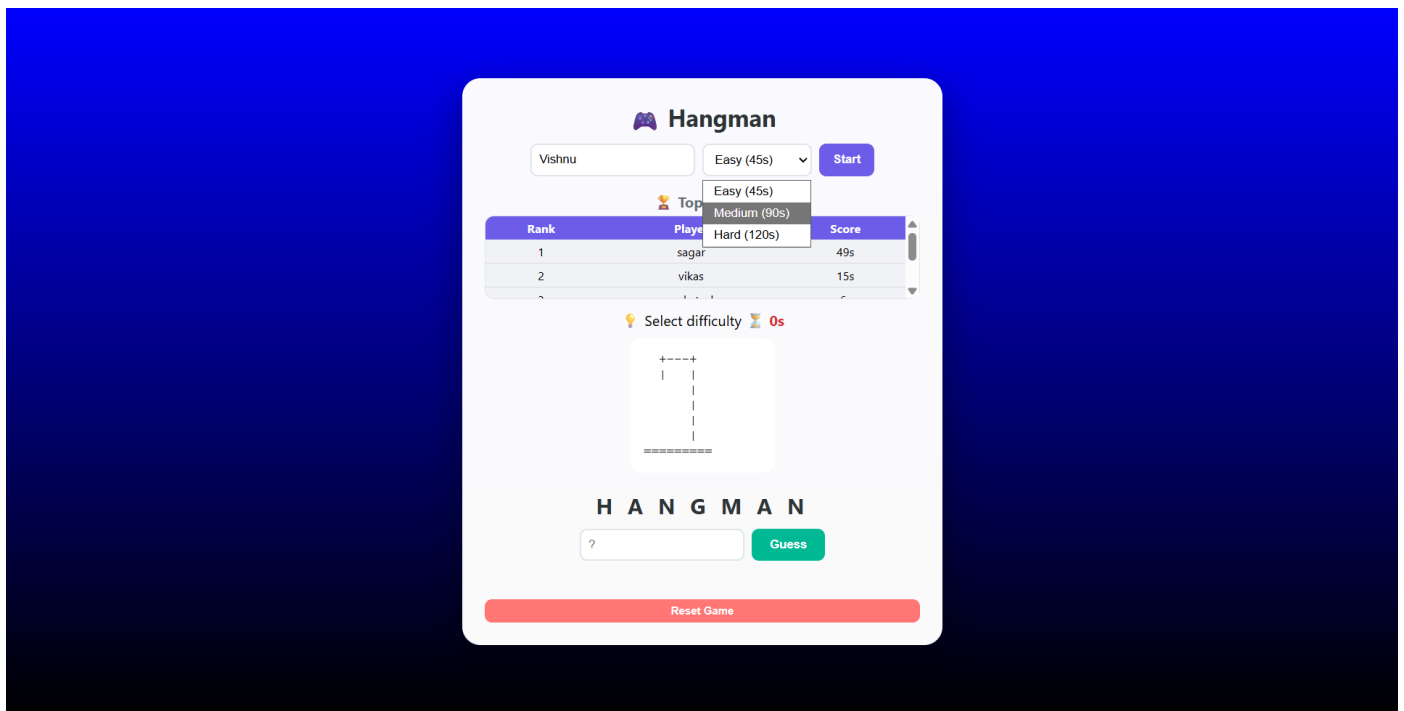
Dashboard:



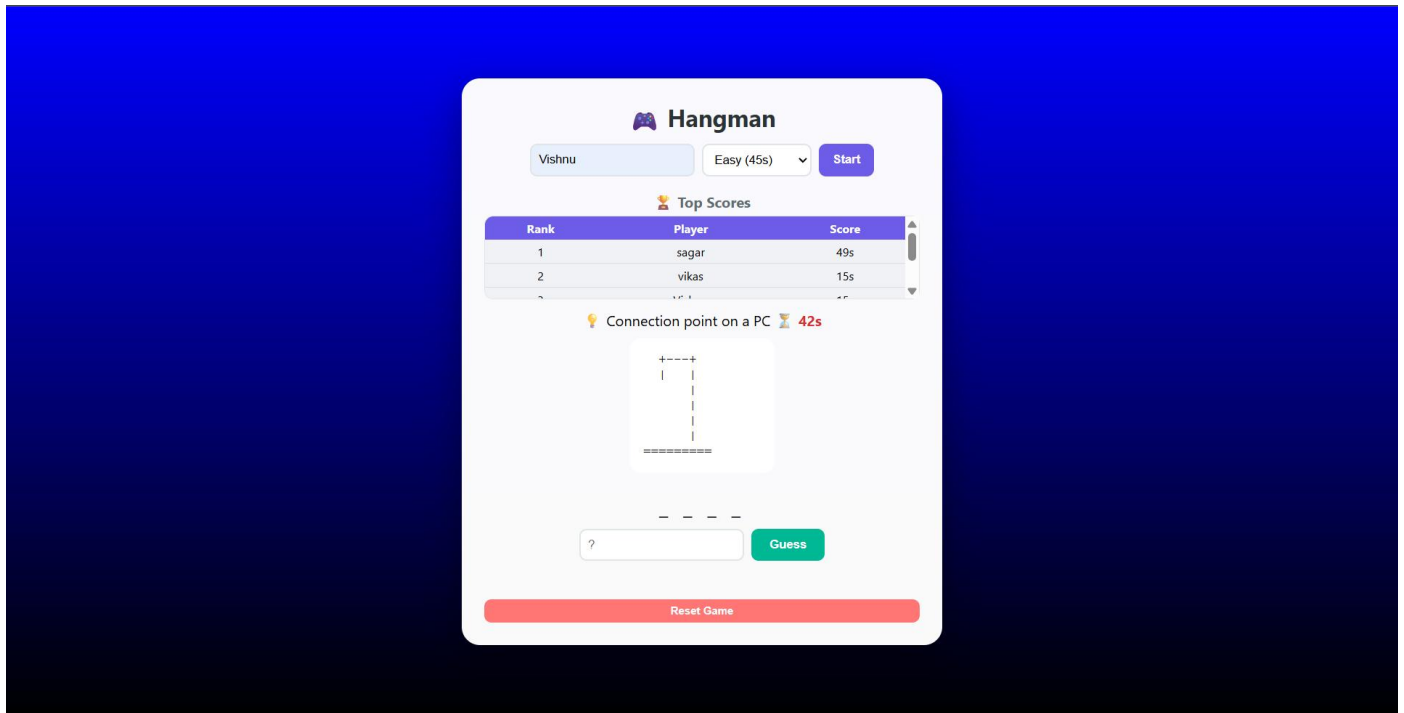
Player Name Setup:



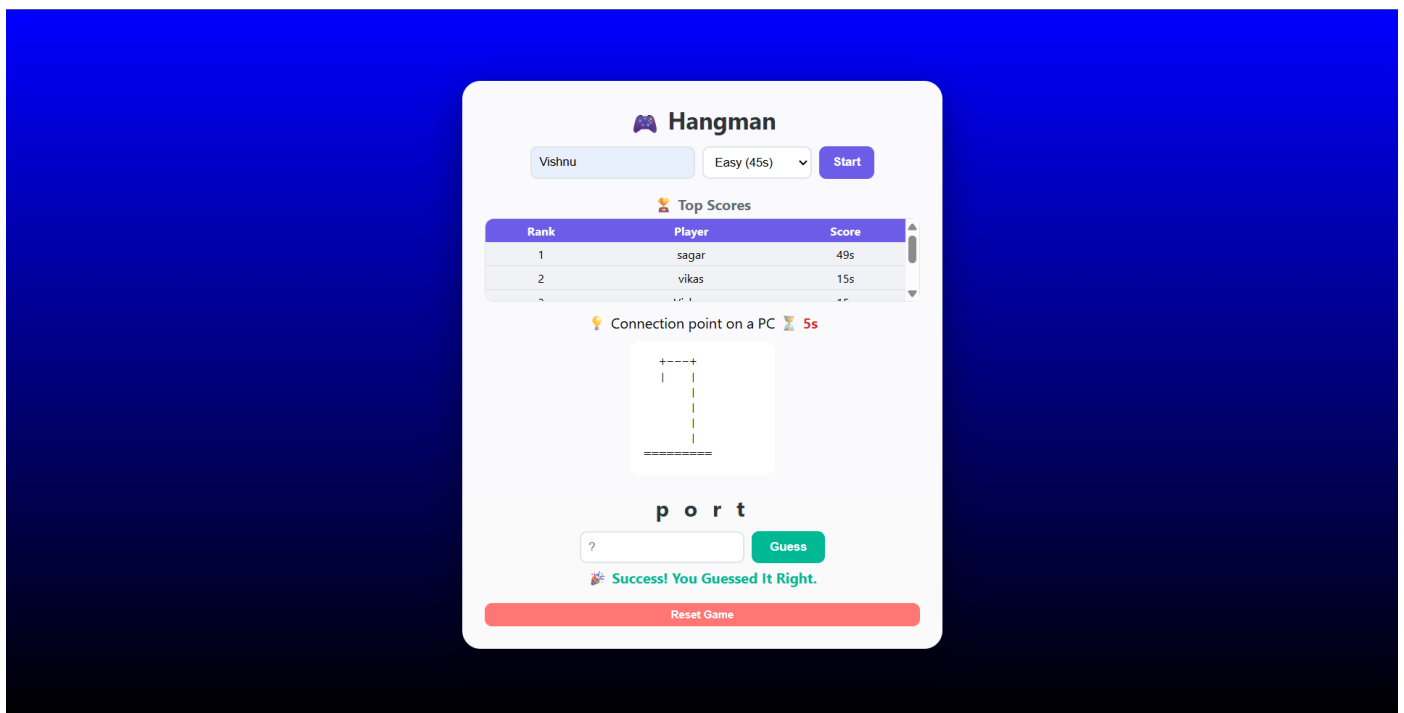
Difficulty Level:

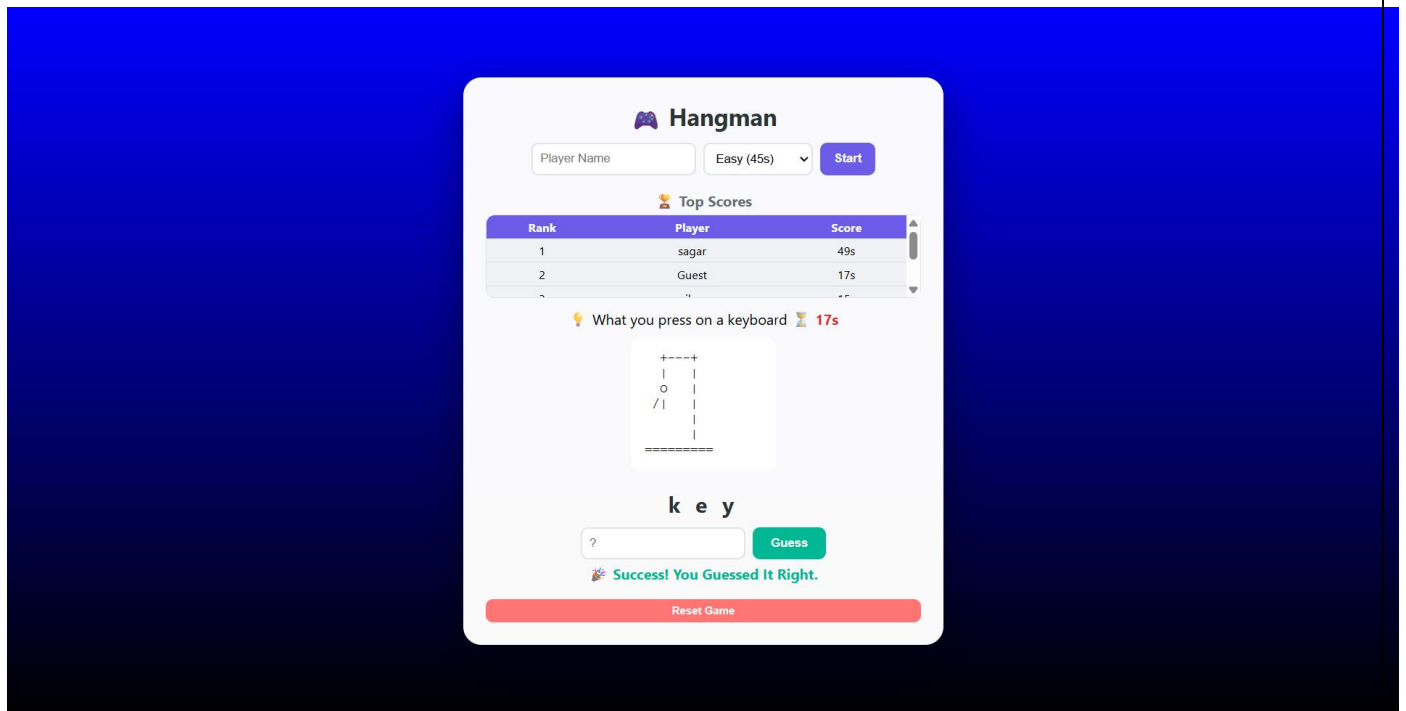


Time Setup:

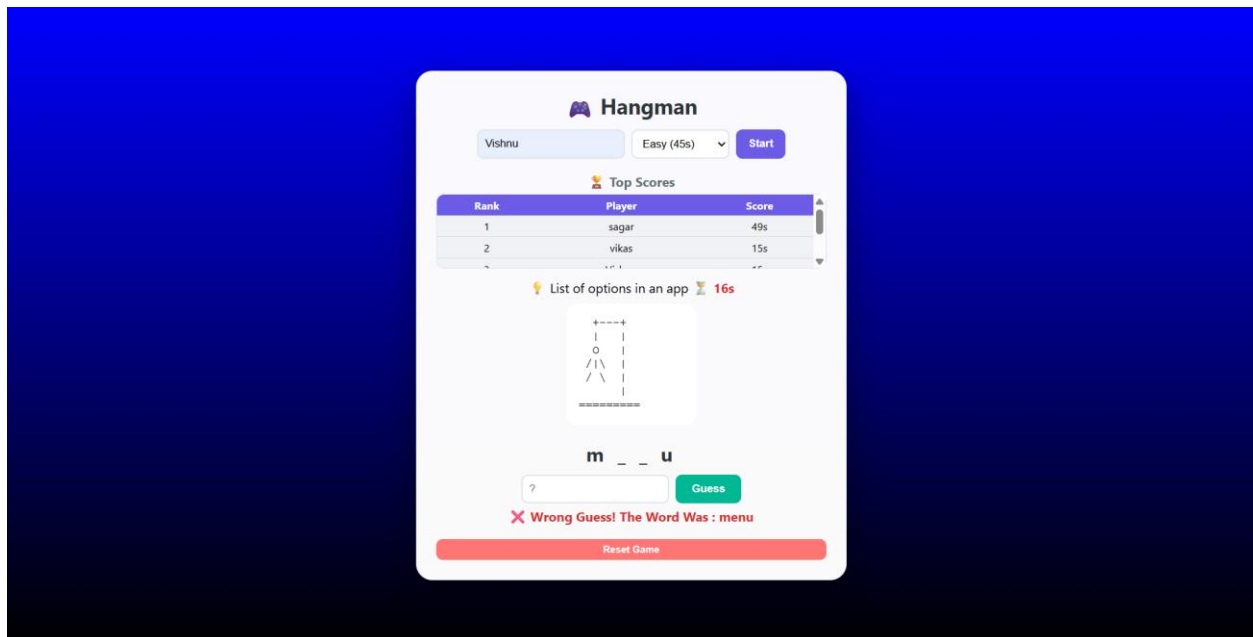


Word Guess:





Wrong Word Guess:



CHAPTER V – CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The Hangman Game Project developed using C language and Web Technologies successfully achieves its objective of providing an interactive and engaging word guessing game experience using fundamental programming concepts. The system allows users to perform essential game operations such as selecting difficulty levels, guessing letters, tracking attempts, managing a countdown timer, and viewing leaderboard scores through a simple and user-friendly interface.

The project effectively demonstrates:

- Use of structures to represent real-world word entries and player records
- Use of arrays for storing multiple word records across Easy, Medium, and Hard difficulty levels
- Implementation of a boolean array for efficiently tracking guessed letters during gameplay
- Application of loops and conditional statements for game control flow and win or loss detection
- Implementation of localStorage in the web version for permanent leaderboard score storage
- Use of timer functions for countdown management in both the C and JavaScript versions
- Application of modular programming through dedicated functions for each game operation

The system performs efficiently for the defined word bank size and is suitable for academic mini-project purposes. However, it has limitations such as a fixed word bank size, no prevention of repeated words across consecutive games, lack of multiplayer support, and a leaderboard restricted to local browser storage only in the web version.

Overall, the Hangman Game Project helps in understanding core C programming concepts such as structures, arrays, functions, string handling, and time management, as well as web development concepts such as DOM manipulation, event handling, and localStorage. It provides practical experience in developing a simple yet complete interactive game application and demonstrates how real-world games can be effectively implemented using fundamental programming knowledge.

5.2 Future Enhancements

The current Hangman Game Project is a basic console-based and browser-based application. In the future, the following improvements can be implemented to enhance performance, usability, and functionality:

Dynamic Word Bank

- Replace static fixed-size word arrays with a dynamically loaded word database
- Removes the fixed word bank size limitation
- Allows unlimited words to be added without modifying the source code
- Words can be loaded from an external text file or online API for endless gameplay variety

Advanced Search and Word Selection

- Implement category-based word selection such as Animals, Technology, Sports, and Movies
- Use randomization algorithms to prevent the same word from appearing in consecutive games
- Add a difficulty rating system based on word length and letter frequency
- Reduces word repetition and improves replayability

Graphical User Interface (GUI)

- Develop an animated GUI version using:
 - Unity or Pygame for a desktop game version
 - React or Vue.js for an enhanced web version
- Add animated hangman drawing instead of ASCII art
- Add visual effects, background themes, and sound effects
- Makes the system more visually appealing and user-friendly

Database Integration

- Connect with a database like:
 - MySQL or MongoDB for backend storage
- Store global leaderboard scores accessible by all players worldwide
- Suitable for large-scale multiplayer applications
- Better data security and score management

Security Features

- Add player login authentication using username and password
- Encrypt stored leaderboard data to prevent tampering
- Add admin role for managing word bank entries and player records
- Implement session management for registered players

Advanced Game Features

- Add multiplayer mode allowing two players to compete against each other
- Add a hint reveal option that costs extra attempts or time penalty
- Implement a scoring system based on word difficulty, remaining time, and number of wrong guesses
- Add a streak system that rewards players for winning consecutive games
- Include an option for players to suggest and submit their own words
- Add sound effects for correct guesses, wrong guesses, win, and game over events

Mobile or Full Stack Web Version

- Convert into a mobile app using React Native or Flutter for Android and iOS platforms
- Develop a full stack web-based Hangman game with a backend server using Node.js or Django
- Implement real-time multiplayer functionality using WebSockets
- Add global leaderboards, player profiles, and achievement badges for a complete gaming experience.